

Principios SOLID en C#

Los principios de diseño SOLID en C# son principios básicos de diseño.

SOLID significa:

- Principio de Responsabilidad Única (SRP)
- Principio Abierto/Cerrado (OSP)
- Principio de Sustitución de Liskov (LSP)
- Principio de Segregación de Interfaces (ISP)
- Principio de Inversión de Dependencias (DIP).

Y en inglés:

- S: Single Responsibility Principle (SRP)
- O: Open-closed Principle (OCP)
- L: Liskov Substitution Principle (LSP)
- I: Interface Segregation Principle (ISP)
- D: Dependency Inversion Principle (DIP)

Principio de Responsabilidad Única (SRP) en C Sharp

Una clase debe tener una sola razón para cambiar, es decir, debe tener una sola responsabilidad o propósito.

Este principio fomenta la creación de clases más específicas que realizan una única tarea bien definida, en lugar de múltiples tareas.

Dividir las clases en unidades más pequeñas y específicas facilita la comprensión, el mantenimiento y las pruebas del código.

Un ejemplo que incumple el SRP:

```
public class User
{
    public string Username { get; set; }
    public string Email { get; set; }

    public void Register()
    {
        // Register user logic, e.g. save to database...

        // Send email notification
        EmailSender emailSender = new EmailSender();
        emailSender.SendEmail("Welcome to our platform!", Email);
    }
}

public class EmailSender
```

```

{
    public void SendEmail(string message, string recipient)
    {
        // Email sending logic
        Console.WriteLine($"Sending email to {recipient}: {message}");
    }
}

```

En este ejemplo, la clase `User` gestiona los datos del usuario (nombre de usuario y correo electrónico) y contiene la lógica para registrarlo.

Esto infringe el Principio de Responsabilidad Única (SRP) porque la clase tiene más de un motivo para cambiar. Podría cambiar debido a:

- Modificaciones en la gestión de datos del usuario, por ejemplo, añadir más campos como nombre, género o aficiones.
- Modificaciones en la lógica de registro de un usuario; por ejemplo, podríamos optar por obtener un usuario de la base de datos mediante su nombre de usuario en lugar de su correo electrónico.

Para cumplir con el Principio de Responsabilidad Única, deberíamos separar estas responsabilidades en clases independientes.

Refactorización del código para satisfacer el SRP:

```

public class User
{
    public string Username { get; set; }
    public string Email { get; set; }
}

public class EmailSender
{
    public void SendEmail(string message, string recipient)
    {
        // Email sending logic
        Console.WriteLine($"Sending email to {recipient}: {message}");
    }
}

public class UserService
{
    public void RegisterUser(User user)
    {
        // Register user logic...

        EmailSender emailSender = new EmailSender();
        emailSender.SendEmail("Welcome to our platform!", user.Email);
    }
}

```

En el código refactorizado, la clase **User** se encarga exclusivamente de representar los datos del usuario. La clase **UserService** ahora gestiona el registro de usuarios, separando así las responsabilidades relacionadas con la gestión de datos de usuario de la lógica de registro. La clase **UserService** se encarga únicamente de la lógica de negocio del registro de usuarios.

Esta separación de responsabilidades se ajusta al Principio de Responsabilidad Única, lo que facilita la comprensión, el mantenimiento y la ampliación del código.

Principio Abierto/Cerrado Open/Close (OPC) en C Sharp

Las entidades de software (clases, módulos, funciones, etc.) deben ser abiertas a la extensión, pero cerradas a la modificación.

Este principio promueve la idea de que el código existente debe poder extenderse con nueva funcionalidad sin modificar su código fuente.

Fomenta el uso de la abstracción y el polimorfismo para lograr este objetivo, permitiendo que el código se extienda fácilmente mediante herencia o composición.

Consideremos un ejemplo de una jerarquía de clases **Shape** que calcula el área de diferentes figuras geométricas.

Inicialmente, esto viola el Principio Abierto/Cerrado (Open/Close), ya que agregar una nueva figura requiere modificar el código existente:

```
public enum ShapeType
{
    Circle,
    Rectangle
}

public class Shape
{
    public ShapeType Type { get; set; }
    public double Radius { get; set; }
    public double Length { get; set; }
    public double Width { get; set; }

    public double CalculateArea()
    {
        switch (Type)
        {
            case ShapeType.Circle:
                return Math.PI * Math.Pow(Radius, 2);
            case ShapeType.Rectangle:
                return Length * Width;
            default:
                throw new InvalidOperationException("Unsupported shape type.");
        }
    }
}
```

En este ejemplo, la clase **Shape** tiene un método, `CalcularÁrea()`, que calcula el área según el tipo de figura. Agregar una nueva figura, como un triángulo, requeriría modificar la clase `Shape` existente, lo que viola el Principio Abierto/Cerrado (OCP).

Para cumplir con el Principio Abierto/Cerrado, debemos diseñar el sistema de manera que permita la extensión sin modificaciones.

Refactoricemos el código usando herencia y polimorfismo:

```
public abstract class Shape
{
    public abstract double CalculateArea();
}

public class Circle : Shape
{
    public double Radius { get; set; }

    public override double CalculateArea()
    {
        return Math.PI * Math.Pow(Radius, 2);
    }
}

public class Rectangle : Shape
{
    public double Length { get; set; }
    public double Width { get; set; }

    public override double CalculateArea()
    {
        return Length * Width;
    }
}
```

En este código refactorizado, definimos una clase abstracta **Shape** con un método abstracto `CalculateArea()`. Las clases de figuras concretas (**Circle y Rectangle**) heredan de la clase `Shape` y proporcionan sus propias implementaciones de `CalculateArea()`.

Agregar una nueva figura, como un triángulo, implicaría crear una nueva clase —extendiendo el código base— que herede de `Shape` e implemente `CalculateArea()`, sin modificar el código existente.

Esto cumple con el Principio de Comportamiento Abierto (OCP) al permitir la extensión sin modificaciones.

Poder añadir funcionalidades sin modificar el código existente significa que no tenemos que preocuparnos tanto por romper el código que ya funciona y por introducir errores.

Seguir el Principio de Control Operacional (OCP) nos anima a diseñar nuestro software de `Shape` que añadamos nuevas funcionalidades solo mediante la adición de código nuevo. Esto nos ayuda a construir software débilmente acoplado y mantenible.

Principio de Sustitución de Liskov (LSP) en C Sharp

Los objetos de una superclase deben poder reemplazarse por objetos de su subclase sin afectar la corrección del programa.

Este principio garantiza que las jerarquías de herencia estén bien diseñadas y que las subclases cumplan con los contratos definidos por sus superclases.

Las violaciones del LSP pueden provocar comportamientos inesperados o errores al sustituir objetos, lo que dificulta la comprensión y el mantenimiento del código.

Consideremos un ejemplo con una clase `Rectangle` y una clase `Square`, que heredan de una clase `Shape` común. Inicialmente, violaremos el LSP al no cumplir con el comportamiento esperado de estas clases. Luego, lo corregiremos para garantizar que se respete el principio.

```
public abstract class Shape
{
    public abstract double Area { get; }
}

public class Rectangle : Shape
{
    public virtual double Width { get; set; }

    public virtual double Height { get; set; }

    public override double Area => Width * Height;
}

public class Square : Rectangle
{
    public override double Width
    {
        get => base.Width;
        set => base.Width = base.Height = value;
    }

    public override double Height
    {
        get => base.Height;
        set => base.Height = base.Width = value;
    }
}
```

Ahora, comprobemos si `Rectangle` calcula su área correctamente:

```
// Program.cs

var rect = new Rectangle();
rect.Height = 10;
rect.Width = 5;

System.Console.WriteLine("Expected area = 10 * 5 = 50.");

System.Console.WriteLine("Calculated area = " + rect.Area);
```

Ejecutando el programa:

```
Área esperada = 10 * 5 = 50.
```

```
Área calculada = 50
```

¡Perfecto!

Ahora, en nuestro programa, la clase Square hereda de la clase Rectangle, o extiende, porque, matemáticamente, un Square es simplemente un tipo especial de Rectangle, donde su altura es igual a su anchura.

Por ello, decidimos que Square extendiera Rectangle; es como decir "un Square es un Rectangle (de un tipo especial)".

Pero mira lo que sucede si sustituimos la clase Rectangle por la clase Square:

```
var rect = new Square();
rect.Height = 10;
rect.Width = 5;

System.Console.WriteLine("Expected area = 10 * 5 = 50.");

System.Console.WriteLine("Calculated area = " + rect.Area);

Expected area = 10 * 5 = 50.

Calculated area = 25
```

¡Vaya! Hemos infringido el Principio de Sustitución de Liskov (PSL): reemplazamos un objeto de la superclase Rectangle por uno de su subclase Square, lo que afectó la corrección de nuestro programa.

Al modelar Square como una subclase de Rectangle y permitir que el ancho y la altura se establezcan de Shape independiente, infringimos el LSP.

Al establecer el ancho y la altura de un Square, este debería conservar su Shape cuadrada, pero nuestra implementación permite esta inconsistencia.

Corrijamos esto para cumplir con el LSP:

```
public abstract class Shape
{
```

```

public abstract double Area { get; }
}

public class Rectangle : Shape
{
    public double Width { get; set; }

    public double Height { get; set; }

    public override double Area => Width * Height;
}

public class Square : Shape
{
    private double sideLength;

    public double SideLength
    {
        get => sideLength;
        set
        {
            sideLength = value;
        }
    }

    public override double Area => sideLength * sideLength;
}

```

```

// Program.cs

Shape rectangle = new Rectangle { Width = 5, Height = 4 };

Console.WriteLine($"Area of the rectangle: {rectangle.Area}");

Shape square = new Square { SideLength = 5 };

Console.WriteLine($"Area of the square: {square.Area}");

```

En este ejemplo corregido, rediseñamos la clase Square para que la longitud de su lado se establezca directamente.

Ahora, un Square se modela correctamente como una subclase de Shape y cumple con el Principio de Sustitución de Liskov.

¿Cómo satisface esto el Principio de Sustitución de Liskov?

Tenemos una superclase, Shape, y subclases, Rectangle y Square.

Tanto Rectangle como Square mantienen el comportamiento esperado de una Shape: podemos sustituir un Rectangle por un Square y el área se calculará correctamente.

Principio de Segregación de Interfaces (ISP) en C Sharp

Los clientes no deberían verse obligados a depender de interfaces que no utilizan.

Este principio fomenta la creación de interfaces de grano fino que contengan únicamente los métodos necesarios para los clientes que las utilizan.

Ayuda a prevenir la creación de interfaces excesivas que obligan a los clientes a implementar métodos innecesarios, lo que resulta en un código más limpio y fácil de mantener.

Consideremos un ejemplo con figuras 2D y 3D, que inicialmente incumple el ISP.

Incumplimiento deñ principio ISP:

```
public interface IShape
{
    double Area();
    double Volume(); // problem: 2D shapes don't have volume!
}

public class Circle : IShape
{
    public double Radius { get; set; }

    public double Area()
    {
        return Math.PI * Math.Pow(Radius, 2);
    }

    public double Volume()
    {
        throw new InvalidOperationException("Volume not applicable for 2D shapes.");
    }
}

public class Sphere : IShape
{
    public double Radius { get; set; }

    public double Area()
    {
        return 4 * Math.PI * Math.Pow(Radius, 2);
    }

    public double Volume()
    {
        return (4.0 / 3.0) * Math.PI * Math.Pow(Radius, 3);
    }
}
```


En este ejemplo, tenemos una interfaz `IShape` que representa figuras tanto bidimensionales como tridimensionales.

Sin embargo, el método `Volume()` resulta problemático para figuras bidimensionales, como `Circle` y `Rectangle`, ya que carecen de volumen.

Esto infringe el principio de seguridad de interfaz (ISP, por sus siglas en inglés), puesto que los clientes (clases que utilizan la interfaz `IShape`) podrían verse obligados a depender de métodos que no necesitan.

```
var circle = new Circle();
circle.Radius = 10;

System.Console.WriteLine(circle.Area());
System.Console.WriteLine(circle.Volume()); // My text editor doesn't flag a problem...

var sphere = new Sphere();
sphere.Radius = 10;

System.Console.WriteLine(sphere.Area());
System.Console.WriteLine(sphere.Volume());
```

Normalmente, si intento llamar a un método en un objeto que no existe, VS Code me indica que estoy cometiendo un error.

Sin embargo, en el ejemplo anterior, al llamar a `circle.Volume()`, VS Code no muestra ningún problema. Y tiene razón, ya que la interfaz `IShape` obliga a `Circle` a implementar el método `Volume()`, aunque los círculos no tengan volumen.

Es fácil ver cómo la violación del principio de supresión de instrucciones (ISP) puede introducir errores en un programa: en el ejemplo anterior, todo parece correcto hasta que ejecutamos el programa y se produce una excepción.

Solucionando problemas con el ISP

```
public interface IShape2D
{
    double Area();
}

public interface IShape3D
{
    double Area();
    double Volume();
}

public class Circle : IShape2D
{
    public double Radius { get; set; }

    public double Area()
    {
        return Math.PI * Math.Pow(Radius, 2);
    }
}
```

```
public class Sphere : IShape3D
{
    public double Radius { get; set; }

    public double Area()
    {
        return 4 * Math.PI * Math.Pow(Radius, 2);
    }

    public double Volume()
    {
        return (4.0 / 3.0) * Math.PI * Math.Pow(Radius, 3);
    }
}
```

En el ejemplo corregido, hemos dividido la interfaz IShape en dos interfaces más pequeñas y específicas: IShape2D e IShape3D.

Cada clase de forma ahora implementa solo la interfaz relevante para su funcionalidad.

Esto cumple con el Principio de Segregación de Interfaces (ISP), ya que **garantiza que los clientes no se vean obligados a depender de métodos que no utilizan**.

Ahora, los clientes pueden depender únicamente de las interfaces que necesitan, lo que fomenta una mejor reutilización del código y mayor flexibilidad.

Principio de Inversión de Dependencias (DIP) en C Sharp

Los módulos de alto nivel no deben depender de los de bajo nivel.

Ambos deben depender de abstracciones.

La inversión de dependencias es la estrategia de depender de interfaces o clases abstractas en lugar de clases concretas.

Este principio promueve el desacoplamiento entre módulos y el uso de interfaces o clases abstractas para definir dependencias, lo que permite un código más flexible y fácil de probar.

Comencemos con un ejemplo que viola el DIP y luego lo corregiremos.

```
public class Engine // Engine is our "low-level" module
{
    public void Start()
    {
        System.Console.WriteLine("Engine started.");
    }
}

public class Car // Car is our "high-level" module
{
    private Engine engine;

    public Car()
```

```
{
    this.engine = new Engine(); // Direct dependency on concrete Engine class
}

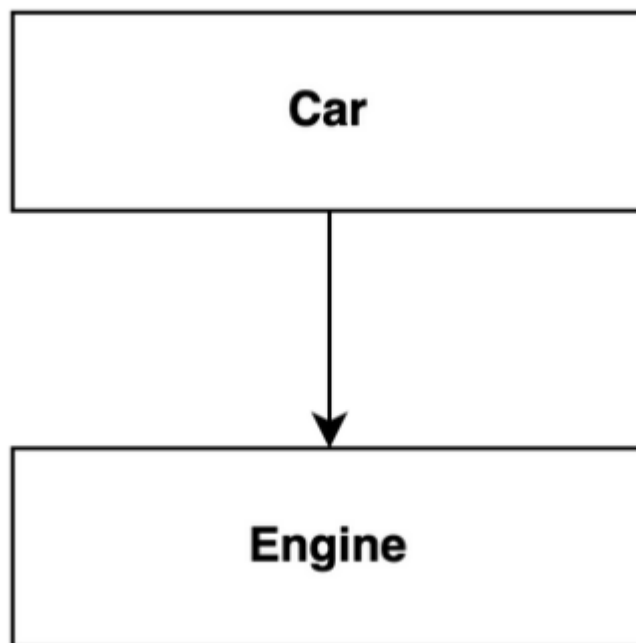
public void StartCar()
{
    engine.Start();
    System.Console.WriteLine("Car started.");
}
}
```

En este ejemplo:

La clase Car crea directamente una instancia de la clase Engine, lo que genera un acoplamiento estrecho entre Car y Engine.

Si la clase Engine cambia, podría afectar a la clase Car, violando el Principio de Inversión de Dependencias.

El diagrama UML que se muestra a continuación ilustra que Car depende de Engine:



¿Pero qué entendemos por clases de “alto nivel” y “bajo nivel”?

Clase de alto nivel: La clase de alto nivel suele representar la funcionalidad principal o la lógica de negocio de la aplicación. Coordina la interacción entre los distintos componentes y suele ser más abstracta.

En este ejemplo, la clase Car puede considerarse la clase de alto nivel. Representa la funcionalidad principal relacionada con el arranque y la conducción del Car. La clase Car se ocupa del comportamiento general del vehículo, como el control de su movimiento.

Clase de bajo nivel: La clase de bajo nivel suele proporcionar funcionalidades o servicios específicos utilizados por la clase de alto nivel. Generalmente se ocupa de los detalles de implementación y es más concreta.

En este ejemplo, la clase Engine puede considerarse la clase de bajo nivel.

Proporciona la funcionalidad específica relacionada con el arranque del Engine.

La clase Engine encapsula los detalles de su funcionamiento, como el encendido y la combustión.

En resumen, la clase Car es la clase de alto nivel, que representa la funcionalidad principal de la aplicación relacionada con el comportamiento del vehículo.

La clase Engine es la clase de bajo nivel, que proporciona una funcionalidad específica relacionada con el funcionamiento del Engine, y que es utilizada por la clase Car.

Aplicando DIP

Para cumplir con el Principio de Inversión de Dependencias, introducimos una abstracción (interfaz) entre el Car y el Engine, lo que permite que el Car dependa de una abstracción en lugar de una implementación concreta.

```
public interface IEngine
{
    void Start();
}

public class Engine : IEngine
{
    public void Start()
    {
        System.Console.WriteLine("Engine started.");
    }
}

public class Car
{
    private IEngine engine;

    public Car(IEngine engine)
    {
        this.engine = engine;
    }

    public void StartCar()
    {
        engine.Start();
        System.Console.WriteLine("Car started.");
    }
}
```

Ahora podemos inyectar cualquier tipo de Engine en las implementaciones de automóviles:

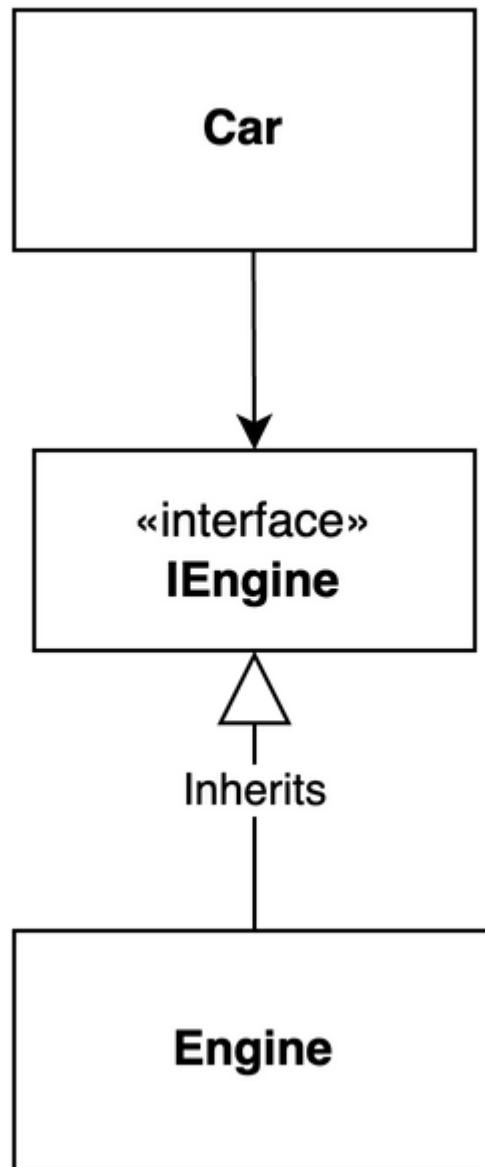
```
var engine = new Engine(); // concrete implementation to be "injected" into the car

var car = new Car(engine);

car.StartCar();
```

En el diagrama UML que se muestra a continuación, podemos observar que ambos objetos ahora dependen del nivel de abstracción de la interfaz.

Engine ha invertido su dependencia con respecto a Car.



En este ejemplo corregido:

1. Definimos una interfaz **IEngine** que representa el comportamiento de un **Engine**.
2. La clase **Engine** implementa la interfaz **IEngine**.
3. La clase **Car** ahora depende de la interfaz **IEngine** en lugar de la clase **Engine** concreta.
4. La inyección de dependencias se utiliza para inyectar la implementación de **IEngine** en la clase **Car**, lo que promueve un acoplamiento flexible.
Ahora, si queremos darle a un coche un tipo de motor diferente, por ejemplo, un motor rápido, podemos inyectarlo en su lugar.
5. Ahora, si la implementación de **Engine** cambia, no afectará a la clase **Car** siempre que cumpla con la interfaz **IEngine**.

La inyección de dependencias (DI) ofrece varias ventajas en el desarrollo de software:

- **Desacoplamiento:** La inyección de dependencias (DI) promueve un acoplamiento flexible entre componentes al eliminar las dependencias directas. Los componentes se basan en abstracciones en lugar de implementaciones concretas, lo que los hace más independientes y fáciles de mantener.

- **Capacidad de prueba:** La inyección de dependencias simplifica las pruebas unitarias al permitir reemplazar fácilmente los componentes con implementaciones simuladas o ficticias durante las pruebas. Esto permite realizar pruebas aisladas de componentes individuales sin depender de sus dependencias.
- **Flexibilidad:** La DI proporciona flexibilidad para configurar e intercambiar dependencias en tiempo de ejecución. Permite usar diferentes implementaciones de dependencias indistintamente sin modificar el código del cliente, lo que facilita la personalización y la extensibilidad en tiempo de ejecución.
- **Legibilidad y mantenibilidad:** Al especificar explícitamente las dependencias en el constructor o los parámetros del método, la DI mejora la legibilidad del código y facilita su comprensión. También reduce el riesgo de dependencias ocultas, lo que resulta en un código más mantenible y comprensible.
- **Reutilización:** La DI promueve la reutilización de componentes al desacoplarlos de sus contextos o entornos específicos. Los componentes pueden diseñarse para ser independientes del marco de trabajo o la plataforma de la aplicación, lo que los hace más portátiles y reutilizables en diferentes proyectos o escenarios.
- **Escalabilidad:* La inyección de dependencias simplifica la gestión de dependencias en aplicaciones a gran escala al proporcionar un enfoque estandarizado para la resolución de dependencias. Ayuda a prevenir el problema de las dependencias complejas y facilita la gestión y el escalado de sistemas complejos.

En resumen, la inyección de dependencias mejora la modularidad, la capacidad de prueba y la mantenibilidad de los sistemas de software, lo que contribuye a una mejor calidad del software y a una mayor productividad de los desarrolladores.

Conclusión

Felicidades! Ahora comprendes los importantísimos principios SOLID.

Estos principios te ahorrarán muchos quebraderos de cabeza durante tu carrera como desarrollador de software y te guiarán hacia la creación de software elegante, mantenible, flexible y comprobable.

Si deseas llevar tus habilidades de desarrollo de software al siguiente nivel y aprender:

Principios de la programación orientada a objetos (POO): encapsulación, abstracción, herencia, polimorfismo, acoplamiento, composición, composición vs. herencia, problema de la clase base frágil.

Los 23 patrones de diseño («Los Cuatro Patrones de Diseño») con ejemplos prácticos.

Lenguaje Unificado de Modelado (UML): la forma estándar de modelar clases y las relaciones entre ellas.

Conclusión

¡Enhorabuena! Ahora comprendes los importantísimos principios SOLID. Estos principios te ahorrarán muchos quebraderos de cabeza durante tu carrera como desarrollador de software y te guiarán hacia la creación de software elegante, mantenible, flexible y comprobable.

Si deseas llevar tus habilidades de desarrollo de software al siguiente nivel y aprender:

- **Principios de la programación orientada a objetos (POO):** encapsulación, abstracción, herencia, polimorfismo, acoplamiento, composición, composición vs. herencia, problema de la clase base frágil.
- **Los 23 patrones de diseño** (Los Cuatro Patrones de Diseño) con ejemplos reales.
- **Lenguaje Unificado de Modelado (UML):** el estándar para modelar clases y las relaciones entre ellas.

Mastering Design Patterns Beginner Friendly Principles

[DesignPrinciplesAndPatterns.pdf](#)

[Design Patterns in C Sharp](#)